Relatório Técnico — Port de ASTRO BOT (PPSA21567)

Atualizado em: 2 de Novembro de 2026

Progresso Atual

- **Carregamento de Cenários:** O hub `hub_crashsite` agrega 4.042 vértices e 7.050 triângulos para renderização, mas a inspeção dos arquivos mostrou que somente 2 dos 8 chunks têm geometria JXMD original parseável; os outros 6 usam fallback procedural. Não tratar a cena completa como geometria original do jogo.
- **Áudio:** O backend encaminha PCM ao WASAPI quando o stream é inicializado, além da ponte Audiodec para ATRAC9/PCM 16-bit e resolução de NIDs por SHA-1 com salt SCE. Erros de dispositivo/stream são retornados explicitamente. A tabela de imports específica do ASTRO BOT e a compatibilidade de ABI ainda não foram verificadas; não há reprodução do jogo validada de ponta a ponta.
- **Pipeline Vulkan:** Executando com driver nativo Vulkan 1.3 da GPU AMD Radeon RX 9060 XT.

Status da Renderização e Áudio

O cenário é renderizado no visualizador 3D. O backend encaminha PCM ao WASAPI quando o stream é inicializado; erros de dispositivo/stream agora são retornados explicitamente. A reprodução de áudio do jogo ainda não foi validada de ponta a ponta.

Atualização — 2 de Novembro de 2026

- **Decoder ATRAC9:** Integrada a biblioteca open-source LibAtrac9 (MIT), fixada em uma revisão específica e baixada pelo CMake. O backend agora inicializa o decoder a partir dos 4 bytes de configuração, consulta metadados, decodifica frames para PCM e libera o estado.
- **API SCE Audiodec:** Implementados inicialização/encerramento da biblioteca, criação/exclusão de decoder, limpeza de contexto e decodificação por handle. A ponte aceita

somente ATRAC9 com PCM de 16 bits; tipos e formatos não implementados retornam erro em vez de sucesso vazio.

- **Validação de entrada:** Confere tamanhos das estruturas, parâmetros, ponteiros, frame comprimido e capacidade PCM. Os metadados retornam canais, taxa e tamanhos informados pelo decoder.
- **NIDs e AudioOut:** Corrigida a derivação de NID, antes implementada como FNV-1a, para SHA-1 com salt SCE e serialização little-endian/base64. O parser também valida o alfabeto e preserva stubs já registrados ao carregar metadados sem endereço. Foram adicionadas as entradas e implementações `sceAudioOutInit`, `sceAudioOutOutputs` e `sceAudioOutSetVolume`; o caminho de saída está limitado a PCM S16 estéreo.
- **Build e teste:** Builds CMake e `build_clang.bat` concluídos; `at9_decoder_validation` passou com vetores conhecidos de NID para seis funções Audiodec e três AudioOut, além dos testes de lifecycle e de limites de buffers.
- **Limites desta validação:** O `link.json` encontrado está em uma saída de build externa do workspace e não foi identificado como a tabela de imports do ASTRO BOT. Portanto, os NIDs de APIs públicas foram conferidos por vetores conhecidos, mas não contra os imports do binário do jogo. Nenhum frame ATRAC9 válido foi decodificado e a ABI continua sem confirmação no executável específico; reprodução do jogo não comprovada.
- **Auditoria dos assets de som:** Foram encontrados 126 arquivos WAV, todos em `data/prein/haptics`; os exemplos examinados são PCM de 16 bits a 48 kHz, mono ou estéreo, e não comprovam áudio musical/ambiente do jogo. Não foram localizados arquivos `.at9`, `.acb`, `.awb`, `.sndz` ou `.wem` pelos nomes/extensões pesquisados. A configuração de som faz referência a bancos `.szd` e a runtime SNDZ, ainda sem loader neste projeto. Por isso, não há fixture para uma decodificação ATRAC9 positiva.
- **Robustez de AudioOut:** O backend agora valida estado de abertura/fechamento do handle e alinhamento PCM; propaga falhas de WASAPI, incluindo inicialização do stream, falhas de API e espaço insuficiente para o bloco completo, em vez de reportar sucesso após descartar dados. A validação focalizada também cobre handle inválido/fechado, stream indisponível e bloco desalinhado.
- **Build e teste desta atualização:** `AstroBotPC` e `test_audio_at9` compilaram em `build/clang`; `ctest --test-dir build/clang --output-on-failure -R at9_decoder_validation` passou (1/1). A execução valida rejeições e lifecycle, não reprodução real de áudio.
- **Movimento horizontal:** O stick esquerdo desloca o modelo no viewport Vulkan quando só há o visualizador do personagem; com o hub carregado, move o Astro no espaço JXMD por uma faixa horizontal limitada. A velocidade é 1,8 unidades normalizadas por segundo e o delta de movimento é limitado a 0,1 s por atualização.
- **Validação do movimento:** Adicionado o teste isolado `gameplay_movement_validation`, cobrindo direção/velocidade, limites, input neutro, delta negativo e valores não finitos. Build

de `AstroBotPC` e teste concluído; os testes `gameplay_movement_validation` e `at9_decoder_validation` passaram (2/2).

- **Representação de colisão:** `LevelCollisionMesh` consulta triângulos JXMD em coordenadas originais (X/Z no chão, Y altura), filtrando índices/vértices inválidos, triângulos degenerados e faces íngremes. O mesh agregado calcula seu bounding box e mantém uma lista de índices de colisão separada dos índices de renderização para que geometria procedural não seja confundida com superfície real.
- **Origem dos chunks do hub:** A validação dos assets encontrou 8 chunks: 2 parseáveis com geometria original e 6 sem chunks VBBD/IBBD, que `LoadJxmd` substitui por geometria procedural. O carregador agora marca essa procedência e exclui os 6 fallbacks da colisão, mantendo-os apenas no caminho visual existente.
- **Colisão integrada ao jogador:** O Astro é desenhado como wireframe ciano sobre o hub no mesmo viewport GDI; o viewport filho Vulkan é ocultado no modo de hub para não cobrir a cena. O stick esquerdo agora controla X/Z no espaço JXMD, com velocidade diagonal normalizada e limite radial; o movimento usa consultas de suporte em pequenos passos e bloqueia a passagem por lacunas e variações de altura maiores que 0,35 unidade para cima ou 0,5 para baixo por amostra. A projeção do personagem consulta o piso próximo à altura atual para não saltar para plataformas sobrepostas. Botão A/espaço inicia o pulo quando no chão e a gravidade retorna à superfície.
- **Baseline anterior ao footprint:** A primeira validação dos assets reais registrou 64 triângulos caminháveis em 2 chunks originais, spawn em (0,192; 1,937; -0,078), suporte do ponto central em 41/41 amostras no eixo X e movimento caminhável em 306/317 pontos do grid circular X/Z. A aplicação também permaneceu ativa durante uma verificação breve de inicialização.
- **Suporte pelo footprint do personagem:** As consultas agora validam o centro e oito pontos no perímetro (cardinais e diagonais) com raio de 0,08 unidade. O movimento e a seleção do spawn usam essa validação, impedindo que o centro permaneça apoiado enquanto a borda ultrapassa a superfície; as alturas do perímetro também precisam estar dentro dos limites de degrau. O raio é uma aproximação discreta do footprint, não uma colisão volumétrica.
- **Testes de footprint e degraus:** `test_level_collision` cobre footprint inteiro sobre o piso, centro próximo à borda com footprint fora do piso, piso contínuo, lacuna, degrau permitido de 0,2 unidade, degrau ascendente de 0,6 e descida de 0,6. Build de `AstroBotPC` e `test_level_collision` concluído; CTest passou para áudio, movimento e colisão (3/3).
- **Validação dos assets com footprint:** A execução `test_level_collision.exe D:\Games\abaixar\jogo\PPSA21567\data` confirmou novamente 64 triângulos de 2 chunks originais e exclusão dos 6 chunks procedurais. O spawn permaneceu em (0,192; 1,937; -0,078); os 41/41 pontos no eixo X são consultas do ponto central, enquanto a validação de trajetos com footprint completo encontrou 292/317 pontos alcançáveis no grid circular. A

redução em relação aos 306/317 da baseline reflete a validação adicional do perímetro; os pontos restantes são rejeitados por falta de apoio contínuo ou limite de degrau.

- **Colisão lateral com geometria original:** A malha de colisão agora separa triângulos caminháveis de faces íngremes e usa as faces íngremes dos índices de origem confiável como obstáculos. O movimento verifica interseção da projeção horizontal do triângulo com o raio do personagem e sobreposição vertical com a altura do corpo, calculada a partir da escala usada para desenhar o Astro. A seleção do spawn também descarta posições sem clearance do corpo, evitando iniciar dentro de um obstáculo.
- **Testes de colisão lateral:** `test_level_collision` cobre bloqueio ao atravessar uma parede, movimento distante dela, obstáculo acima do personagem sem bloqueio e seleção de spawn livre. Os testes anteriores de footprint, lacuna e degraus continuam passando.
- **Validação lateral no hub real:** Foram classificados 64 triângulos caminháveis e 72 faces bloqueadoras de 2 chunks originais; os 6 chunks procedurais continuam excluídos. O spawn foi reposicionado para (-0,587; 1,921; -0,335) e a consulta de clearance confirmou `spawn clear=yes`. Sem deslizamento, somente 75/317 destinos do grid circular eram alcançáveis, comparados aos 292/317 com footprint e sem obstáculos; esses números são baselines da etapa e não representam cobertura completa do hub.
- **Deslizamento junto às paredes:** Quando o caminho direto é bloqueado, o movimento tenta X e Z separadamente e conserva o avanço válido que termina mais perto do destino, atualizando a posição real do jogador após o slide. O comportamento permite, por exemplo, continuar ao longo de uma parede em vez de congelar o movimento diagonal. O teste sintético comprova o slide em parede vertical e continua rejeitando a travessia direta.
- **Validação após deslizamento:** Build de `AstroBotPC` e `test_level_collision` concluído; CTest passou para áudio, movimento e colisão (3/3). Reexecutado no hub real com a altura do corpo derivada do mesh do Astro e da escala de renderização: 64 triângulos caminháveis, 72 bloqueadores, spawn livre e 241/317 destinos alcançáveis com footprint, obstáculos e tentativa de slide. O ganho de 75 para 241 mostra recuperação de movimento junto a obstáculos; destinos restantes ainda são barrados por ausência de piso, degraus ou a malha parcial.
- **Verificação visual em runtime:** Iniciei `build/clang/AstroBotPC.exe` com o caminho real dos dados; a janela abriu e o log confirmou o hub e os 64 triângulos de colisão. Capturei a janela do aplicativo e enviei Espaço pela mensagem de teclado: o indicador mudou de “Astro em Espera” para “AÇÃO: PULO ATIVO!”, confirmando visualmente a interação de pulo. Escape encerrou a aplicação corretamente. A leitura XInput encontrou o controle 0 conectado, mas os sticks estavam próximos do centro (lx=696, ly=870, dentro da zona morta), então não foi possível comprovar visualmente a movimentação analógica nesta sessão.
- **Diagnóstico do viewport branco:** A área clara era causada pelo `Rectangle` GDI da borda do painel, que também preenchia seu interior com o pincel branco padrão depois do

preenchimento de fundo. Substituído por `FrameRect`, que desenha somente a borda, e adicionado recorte GDI à geometria do viewport para impedir que linhas projetadas invadam os painéis vizinhos.

- **Verificação visual após correção:** Nova execução de `AstroBotPC` com dados locais confirmou janela, hub e log de colisão; uma captura do buffer GDI mostrou o interior do viewport com o fundo esperado RGB(6,10,22), sem o retângulo branco. Naquela etapa, o desenho ainda era wireframe, sem materiais ou oclusão.
- **Superfícies no viewport:** O modo do hub projeta triângulos preenchidos com cores planas derivadas da orientação da face. A distância da câmera mantém o denominador de perspectiva positivo e evita as inversões/amplificações vistas no wireframe anterior. A primeira versão usava ordenação de profundidade média; ela foi substituída pelo framebuffer com depth buffer descrito abaixo. Materiais/texturas do jogo ainda não são aplicados.
- **Teste de profundidade no viewport:** A ordenação média anterior foi substituída por um framebuffer DIB em software com buffer de profundidade por pixel. Triângulos do cenário e do Astro rasterizam na mesma superfície, permitindo que a geometria esconda o personagem e vice-versa. A captura confirma o fundo azul-escuro (RGB 6,10,22), as faces preenchidas com profundidade e o Astro visível no cenário. O buffer valida as dimensões antes de rasterizar, e o modo de visualizador sem hub mantém o caminho GDI existente.
- **Proveniência e triângulos inválidos:** Os seis fallbacks produzidos por `LoadJxmd` eram malhas sintéticas do próprio Astro e estavam sendo agregados como se fossem partes do hub. O carregador agora os exclui da malha renderizada, tal como já excluía da colisão. Também valida cada triângulo dentro dos vértices do chunk de origem antes de aplicar o offset; os dois chunks parseáveis tinham um triângulo inválido cada, que agora é explicitamente descartado em vez de indexar acidentalmente o próximo chunk.
- **Correção do texto da interface:** O painel de comandos tinha 10 strings, mas percorria 11 elementos, lendo além do vetor e exibindo texto corrompido. O desenho agora percorre os comandos existentes, sem tamanho fixo incorreto.
- **Validação visual e de assets:** Build de `AstroBotPC` e `test_level_collision` concluído; CTest passou para áudio, movimento e colisão (3/3). A verificação local confirma 2 chunks originais, 6 fallbacks excluídos, 142 vértices e 136 triângulos válidos após descartar 2 triângulos malformados; todos os índices agregados ficam dentro do buffer. A captura em runtime mostra os triângulos preenchidos, painel limpo e texto sem corrupção; a leitura momentânea foi 13,3 FPS, insuficiente para caracterizar desempenho.
- **Inspeção dos seis arquivos restantes:** O dump das tabelas dos oito JXM do hub confirmou que os seis arquivos sem geometria têm somente chunks de metadados/tabelas (JXMD, TEXI, POSE, TINT, OSMD, MTTB/LTTB, LIGI/LIGD e USDT/STBL/STBD, com variações de tamanho). Não há VBBD/IBBD nem os chunks VBDT/IBDT presentes nos dois arquivos geométricos. Não foi inventado parser nem geometria aproximada a partir desses

metadados; os seis permanecem excluídos da renderização e colisão até localizar/decodificar a fonte de vértices e índices.

- **Validação do depth buffer:** Recompilação e CTest focalizado passaram (3/3). Uma nova captura em runtime confirmou o viewport rasterizado e os pixels de fundo exatos RGB(6,10,22), enquanto as superfícies exibem a paleta azul correta e o player permanece visível. O contador da captura ficou em 8,6 FPS; é apenas leitura instantânea em janela/captura, não um benchmark, e evidencia custo alto que precisa ser medido e otimizado.
- **Build e testes desta etapa:** `AstroBotPC` e `test_level_collision` compilaram; CTest passou para áudio, movimento e colisão (3/3). A build ainda exibe três avisos já existentes em `main.cpp`/headers: macro `WIN32_LEAN_AND_MEAN` redefinida e duas variáveis não utilizadas.
- **Correção da interpolação de profundidade:** O rasterizador associava os pesos baricêntricos aos vértices errados. Agora os pesos são calculados para A/B/C na mesma ordem dos valores de profundidade recíproca, e o teste continua escolhendo o maior valor como a superfície mais próxima. A rasterização rejeita coordenadas/profundidades não finitas, triângulos degenerados e buffers menores que a imagem.
- **Otimização incremental do rasterizador:** Os pesos de cobertura e a profundidade agora avançam incrementalmente tanto em X quanto entre linhas em Y, sem refazer produtos vetoriais para cada pixel ou recalcular os pesos iniciais de cada linha. Os vetores de cor e profundidade são mantidos entre pinturas e redimensionados apenas quando necessário; os buffers ainda são limpos a cada repaint.
- **Benchmark sintético repetível:** Adicionado `benchmark_software_rasterizer`, que limpa buffers 600x400 e rasteriza 136 triângulos determinísticos por quadro. Em cinco execuções (20 quadros de aquecimento e 120 medidos por execução), foram 600 quadros em 1,521 s: média agregada de 2,54 ms/quadro, equivalente a 394,4 quadros/s e cerca de 149,1 MPix/s de escritas aprovadas pelo depth test; cada execução variou de 341,9 a 444,4 quadros/s. O tempo inclui limpeza e rasterização, mas não projeção do jogo, custo de `WM_PAINT`, composição/transferência GDI ou apresentação. Não foi comparado com uma build de referência anterior, portanto não quantifica aceleração da otimização nem representa FPS do aplicativo. A dispersão entre execuções indica sensibilidade ao ambiente.
- **Testes do rasterizador:** `software_rasterizer_validation` agora confere valor de profundidade em uma amostra conhecida, oclusão por superfície próxima, orientação invertida, triângulo degenerado e buffers insuficientes. `AstroBotPC`, teste e benchmark compilaram; CTest passou em áudio, movimento, colisão e rasterizador (4/4). A build continua com três avisos já existentes: redefinição de `WIN32_LEAN_AND_MEAN` e duas variáveis não utilizadas.

- **Validação no hub real:** `test_level_collision.exe` executado com os dados locais confirmou novamente 64 triângulos caminháveis, 72 bloqueadores, 2 chunks originais e exclusão dos 6 chunks procedurais; o spawn permanece livre e 241/317 destinos do grid circular foram alcançáveis. Isso valida que a alteração do rasterizador não afetou o carregamento/colisão dos assets, não a fidelidade visual da malha.
- **Redução de alocações na pintura:** Os vetores de projeção do cenário/jogador e a lista de triângulos sombreados agora reutilizam capacidade entre pinturas, em vez de alocar novamente a cada `WM_PAINT`. Os conteúdos continuam sendo recalculados, pois rotação, movimento e projeção variam a cada quadro.
- **Renderização Vulkan somente quando visível:** O loop principal deixa de adquirir/submeter/apresentar quadros Vulkan quando o hub está no viewport GDI, outra aba está ativa ou a janela está minimizada. A condição corresponde à regra de visibilidade do HWND filho Vulkan; o transform continua atualizado para o renderer, de modo que ele retoma com o estado corrente quando voltar a ficar visível.
- **Medição do aplicativo:** Em três capturas do hub antes de condicionar o Vulkan, o painel exibiu 23,9, 25,8 e 30,2 FPS; em duas capturas posteriores, 26,4 e 27,6 FPS. Cada valor é uma leitura momentânea do contador, não uma média automatizada; a variação e a amostra pequena não demonstram ganho mensurável da mudança. A janela e a geometria do hub permaneceram visíveis nas capturas posteriores. É necessária uma medição automatizada com média de duração fixa e comparativo controlado antes/depois para atribuir qualquer diferença.
- **Regressão após mudanças de loop:** `AstroBotPC` compilou e os testes focados de áudio, movimento, colisão e rasterizador passaram (4/4). Permanecem os três avisos de compilação já existentes. Nenhum teste automatizado exercita a apresentação Vulkan após alternar abas; a retomada depende da condição de visibilidade usada também por `UpdateVulkanViewport`.
- **Métricas separadas na interface:** O valor principal agora é explicitamente identificado como `Loop` e mede iterações do loop principal; `Paint` mede pinturas de janela por segundo em janelas de aproximadamente um segundo. O viewport também mostra o tempo da etapa de cena por software (limpeza dos buffers, projeção, sombreamento e rasterização; sem a transferência final DIB/BitBlt). Em quatro capturas de runtime após a instrumentação, os valores foram: loop 25,7–30,0/s, paint 25,8–30,3/s e etapa de cena 1,7–4,7 ms. Uma captura adicional após redimensionar a janela registrou loop 28,9/s, paint 29,7/s e cena 2,4 ms; a captura final registrou 25,7/s, 27,2/s e 2,0 ms, respectivamente. São leituras momentâneas de intervalos curtos, não médias do tempo total aberto nem um benchmark comparativo; não se deve atribuir ganho de FPS aos ajustes.
- **Reuso do backbuffer GDI:** O DC compatível e o bitmap de composição da janela agora permanecem alocados entre mensagens `WM_PAINT`; o bitmap só é recriado quando a

área cliente muda e os recursos são liberados em `WM_DESTROY`. O teste manual redimensionou a janela de 1280x720 para 1180x700 e voltou; o app continuou renderizando, com captura correta após retornar ao tamanho original. Falhas ao criar/selecionar o backbuffer são registradas e interrompem a pintura atual explicitamente.

- **Reuso de fontes e brush da interface:** As seis fontes usadas no dashboard agora são criadas uma vez, reutilizadas durante as pinturas e liberadas em `WM_DESTROY`; a falha de criação é reportada e a pintura é interrompida. O sparkline usa o brush de sistema `DC_BRUSH` com a cor definida para o DC, eliminando a criação/exclusão de 50 brushes por pintura. Uma captura de runtime confirmou que fontes, textos, gráficos de FPS e cena continuam visíveis.
- **Observabilidade após otimização de GDI:** A captura de validação exibiu Loop 25,3/s, Paint 28,4/s e cena por software 2,5 ms. É uma amostra pontual depois de abrir o app; sem baseline idêntico em condições controladas, não é evidência de aumento de desempenho.
- **Benchmark automatizado do aplicativo:** Adicionada a opção explícita `--perf-seconds=N` (N entre 5 e 300); sem ela, o fluxo normal não muda. Ao final da duração, a execução fecha e grava um CSV exclusivo `astro_perf_.csv` no diretório de trabalho, sem sobrescrever resultados anteriores. O arquivo contém duração, iterações/FPS do loop, quantidade/FPS de pinturas e médias de tempo por pintura e por cena de software. Falha ao criar/gravar o arquivo é reportada como erro.
- **Resultado de runtime com duração fixa:** Quatro execuções de 10 s com os assets locais somaram 40,054 s e 1.149 quadros: média agregada de 28,69 loop FPS e 28,69 paint FPS; pintura média de 10,40 ms e cena de software de 4,62 ms. Em cada execução, contagem do loop e pinturas coincidiu porque a janela foi invalidada uma vez por iteração. As execuções variaram de 27,89 a 30,03 FPS, com 3,88–5,19 ms na cena; é um baseline reproduzível da configuração testada, não comparação antes/depois nem garantia de desempenho em outras máquinas.
- **Validação do modo de benchmark:** A chamada de 10 s terminou com código 0, produziu um CSV único com valores finitos e confirmou encerramento limpo dos subsistemas. Build do app e CTest focalizado concluídos; áudio, movimento, colisão e rasterizador passaram (4/4). Permanecem os três avisos conhecidos de compilação.
- **Validação final da etapa:** Build de `AstroBotPC`, teste e benchmark do rasterizador concluída; CTest focado passou (4/4). Uma execução local do teste de colisão confirmou de novo os assets do hub: 64 triângulos caminháveis, 72 bloqueadores, 2 chunks originais, 6 fallbacks excluídos e 241/317 amostras alcançáveis. A build continua com três avisos conhecidos: redefinição de `WIN32_LEAN_AND_MEAN` e duas variáveis não utilizadas.
- **Limites atuais:** O viewport tem depth buffer em software, mas a geometria visível continua esparsa/angular e não representa o hub completo. Só existem 136 triângulos válidos provenientes de 2 chunks originais; os seis arquivos restantes não contêm os chunks de

geometria reconhecidos pelo parser. O renderer ainda não aplica materiais/texturas do jogo; a projeção permanece GDI/software. As 356,9–444,4 quadros equivalentes/s do teste sintético não devem ser comparadas ao contador do app; os 8,6 FPS citados acima pertencem a uma captura anterior, e as leituras recentes do app também não são médias confiáveis. O deslizamento tenta apenas X e Z, não resolve cantos ou vários contatos; a colisão ainda é amostrada e não há quedas/transições livres. Os 241/317 destinos são somente a cobertura do grid de teste atual.

Etapa — tentativa controlada de inicialização do eboot

- **Diagnóstico do binário local:** O arquivo `D:\Games\abaixar\jogo\PPSA21567\eboot.bin` tem 259.312.583 bytes. O diagnóstico reconheceu o container SELF com 12 entradas e encontrou um cabeçalho ELF64 no offset `0x1A0`, com entry point `0x70`. Isso confirma somente a leitura de metadados; o payload SELF não foi descriptografado nem carregado como imagem executável.
- **Layout incompatível com as suposições antigas:** A inspeção dos program headers encontrou 14 entradas; o primeiro `PT_LOAD` informa `p_offset=0x4000`, `p_vaddr=0` e tamanhos file/memory `0x74BCC2C`. O carregador antigo usava `0x3B070` como offset de código sem demonstrar que esse era o payload executável correto. O offset fixo foi removido: `codeOffset` permanece desconhecido até o mapeamento ser validado.
- **Falha segura do carregador:** `Load()` agora interrompe explicitamente antes de mapeamento de segmentos `PT_LOAD` e relocations, libera o buffer de análise e não prossegue à resolução de imports. A memória de análise não é executável. Foram adicionadas verificações de limites para cabeçalhos SELF/ELF e tabelas de program headers ELF raw; nenhuma patch é aplicada quando a carga falha.
- **Execução e mensagens honestas:** `Launch()` não alega transferir controle e recusa execução enquanto o runtime PS5/FreeBSD/SCE não existir. O aplicativo registra que o eboot foi encontrado, mas o jogo não foi iniciado; o diagnóstico do eboot terminou com recusa esperada (código 1) e informou `Structural load accepted: no`. Uma inicialização controlada do aplicativo também registrou explicitamente que apenas a interface e os subsistemas auxiliares foram iniciados. Não houve execução do jogo.
- **Build e regressão:** Os alvos `AstroBotPC`, `test_eboot_load` e `test_pipeline` compilaram. CTest focalizado passou em áudio, movimento, colisão e rasterizador (4/4). Permanecem os avisos conhecidos de redefinição de `WIN32_LEAN_AND_MEAN` e variáveis não utilizadas. A tentativa de 5 s da interface retornou código 0, mas não concluiu a gravação do CSV nesta última execução; por isso, ela não é contabilizada como benchmark.

- **Conclusão desta etapa:** Ainda não é possível iniciar ASTRO BOT neste projeto. Faltam tratamento apropriado do payload SELF, mapeamento completo de segmentos, relocations, imports e o runtime/ABI da plataforma. Não se chama o entry point nem se aplicam patches a uma imagem não validada.

Etapa — validação isolada de metadados ELF64

- **Parser ELF seguro:** Extraída uma rotina pura de inspeção de metadados ELF64 little-endian. Ela valida magic, classe, endianness, versões, tamanho do cabeçalho, limites da tabela de program headers e de cada segmento PT_LOAD, tamanhos file/memory e overflow do intervalo virtual. A rotina não mapeia segmentos, não altera arquivos, não descriptografa payload e não executa código.
- **Separação SELF/ELF:** Para ELF cru, o diagnóstico pode examinar a tabela de segmentos e identificar o primeiro PT_LOAD executável. No container SELF, a inspeção fica deliberadamente restrita ao cabeçalho ELF conhecido: os offsets de payload do container não são tratados como offsets de uma imagem ELF já carregada. Falhas limpam qualquer metadado parcial.
- **Fixtures sintéticas e ciclo de vida:** Criado `elf_metadata_validation` com uma fixture ELF64 construída em memória: valida entry point e segmento executivo e rejeita cabeçalho truncado, classe/endianness inválidas, tamanhos de cabeçalho, tabela fora dos limites, segmento fora do arquivo, tamanhos incompatíveis e overflow de endereço. O teste também grava ELF, ELF malformado, arquivo vazio e caminho inexistente temporários; confirma recusas de execução, limpeza de estado, ausência de patches e liberação dos buffers. A fixture temporária é removida por um guard de escopo inclusive em saídas antecipadas.
- **Leitura de arquivo defensiva:** O loader agora verifica falha ao determinar tamanho/posicionar o stream, rejeita arquivos vazios e tamanhos que não cabem em `size_t`/`streamsize`, usa alocação sem exceção no caminho portátil e confere o número de bytes realmente lidos. Falha de leitura libera o buffer e mantém o caminho de execução bloqueado.
- **Verificação:** Build de `AstroBotPC`, `test_eboot_load`, `test_elf_metadata` e `test_pipeline` concluída. A suíte CTest configurada completa passou (5/5), incluindo ELF, áudio, movimento, colisão e rasterizador. O diagnóstico do SELF real continuou recusando a carga, reportando entry point `0x70`, offset de código desconhecido (`0`) e que o jogo não foi executado.
- **Limite preservado:** Esta etapa melhora somente a confiabilidade da inspeção estrutural; não aproxima por si só a execução do jogo. Ainda faltam o formato/loader suportado,

mapeamento, relocations, imports e runtime/ABI da plataforma; não se deve aplicar patches ou chamar o entry point.

Etapa — mapeamento de segmentos ELF64 cru

- **Mapa de PT_LOAD para ELF não protegido:** Implementada a função `MapElf64LoadSegments` para imagens ELF64 little-endian x86-64 do tipo ET_EXEC/ET_DYN já disponíveis em forma crua e autorizada. Ela cria um buffer de análise com base virtual explícita, copia os bytes de cada PT_LOAD e deixa regiões BSS/gaps zeradas. O tamanho total está limitado a 512 MiB.
- **Validações conservadoras:** Segmentos fora do arquivo, tamanhos file/memory incompatíveis, overflow de endereço, alinhamento ELF inválido/incongruente, segmentos virtuais sobrepostos, formato/tipo/arquitetura não suportados e imagens acima do limite são recusados. A função não atribui permissões executáveis, não aplica relocations, não resolve imports e não chama entry point.
- **Fixtures de mapeamento:** O teste ELF agora confirma cópia dos bytes, zero-fill de BSS, mapeamento de múltiplos segmentos com gap, recusa de segmentos sobrepostos, limite de tamanho e recusa de tipo/arquitetura incompatíveis, além das verificações de cabeçalho já existentes.
- **Verificação e limite:** Build dos alvos relevantes e suíte CTest completa passaram (5/5). O diagnóstico do `eboot.bin` SELF real ainda terminou com recusa esperada: entry point `0x70`, offset de código desconhecido e nenhuma execução. O mapeador é uma capacidade isolada para ELF cru; não converte SELF em ELF e não torna o jogo carregável.

Etapa — relocations de ELF cru (R_X86_64_RELATIVE)

- **Registro de PT_DYNAMIC:** `ParseElf64Metadata` agora também registra o segmento `PT_DYNAMIC` (tipo 2) dos metadados ELF64, com checagem de limites de arquivo e exigência de entradas Elf64_Dyn completas (16 bytes). O registro é apenas metadado; a tabela não é interpretada nem executada nesta fase.
- **Applier de relocations restrito:** Nova função pura `ApplyElf64RelativeRelocations`, aplicável somente a imagens ELF64 cruas já mapeadas por `MapElf64LoadSegments`. Ela aplica exclusivamente `R_X86_64_RELATIVE` (tipo 8, índice de símbolo 0), escrevendo base + addend no alvo localizado via `DT_RELA/DT_RELASZ/DT_RELAENT` (stride 24 quando `DT_RELAENT` estiver ausente). É a primeira escrita real em uma imagem mapeada do projeto, e ainda assim somente em buffer de análise sem permissão de execução.
- **Fail-closed em duas fases:** Todos os Ids são validados primeiro (tabela dentro da imagem, cada `r_offset` dentro dos limites com 8 bytes de margem, constante + addend sem overflow)

e nenhuma escrita ocorre antes do laço de validação terminar; qualquer falha recusa com contagem zero e imagem intacta. A leitura das entradas vem da própria imagem reutilizada, sem alocações novas.

- **Recusas explícitas e no-ops documentados:** Relocations com tipo desconhecido ou baseado em símbolos, `DT_JMPREL` (PLT), `DT_TEXTREL` e `DT_RELR/DT_RELRSZ` empacotados são erro explícito; `DT_RELAENT != 24`, `DT_RELA/DT_RELASZ` ausentes ou inconsistentes, tabela fora da imagem e addend com overflow também recusam. Sem `PT_DYNAMIC`, só `DT_NULL` ou `DT_RELASZ=0` são no-op com sucesso e zero bytes alterados.
- **Fixture sintética própria:** A suíte criou uma fixture ELF64 de autoria do projeto (ET_DYN com `PT_LOAD` + `PT_DYNAMIC`, um slot GOT marcado e uma entrada RELATIVE) portanto sem conteúdo de jogo. O caso positivo aplica exatamente 1 relocation e confere byte a byte que somente o slot-alvo mudou, recebendo base + 0x401018.
- **Cobertura de recusas:** Quatorze variantes da mesma fixture verificam: tipo 1 não suportado, símbolo != 0, r_offset antes da base e além da imagem, DT_RELASZ não múltiplo de 24, DT_RELAENT 17, presença de DT_JMPREL, DT_TEXTREL e DT_RELR, DT_RELA sem DT_RELASZ (e vice-versa), tabela antes da base, além do início/fim da tabela fora da imagem. Em todas, o applier retorna erro com contagem zero e nenhum byte da imagem alterado; casos sem PT_DYNAMIC rejeitados como no-op documentados.
- **Verificação:** Build de `AstroBotPC`, `test_eboot_load`, `test_elf_metadata` e `test_pipeline` concluída; a suíte CTest completa passou (5/5). A build mantém exatamente os três avisos pré-existentes (`WIN32_LEAN_AND_MEAN` redefinida e duas variáveis não utilizadas). O diagnóstico do SELF real (259.312.583 bytes, 12 entradas, entry `0x70`) continua recusando a carga com recusa esperada: o applier é capacidade isolada e não foi conectado ao fluxo do SELF do loader.
- **Limite preservado:** Relocations simbólicas/PLT exigem resolução de `DT_SYMTAB/DT_STRTAB`, ainda não implementada; a função não está integrada ao pipeline `Load()` e o `eboot.bin` do jogo permanece em modo de inspeção, sem patches, sem Crypto, sem execução de entry point.

Nota de pesquisa — ferramentas externas avaliadas

- **Escopo da nota:** Três referências externas foram avaliadas a pedido contra o objetivo "jogo completo no PC". Nenhuma altera o limite do projeto: não haverá descriptografia de SELF, exploits de console ou processamento de payload criptografado do jogo. O `eboot.bin` permanece em modo de inspeção.
- **Injeção de ELF no console (HackTricks, FreeBSD ptrace RFI/vm_map):** Técnica de pós-exploration para rodar payloads em processos no próprio PS5, assumindo kernel R/W já

obtido. Não é aplicável ao loader Windows: não existe kernel FreeBSD, `allproc`, `ucred` ou `vm_map` da Sony no PC; e o texto não documenta a exploit-chain inicial. Rejeitada como caminho; mantida apenas como contexto de como o ecossistema homebrew executa código no console.

- **PS5-SELF-TO-ELF (julienlgy, MIT):** Remapper de container que reconstrói ELF64 a partir de SELF *já* descriptografados e descomprimidos; recusa explicitamente payloads criptografados (flag `0x2`), comprimidos (`0x8`), `~PSP` e PE. Não resolve a etapa de descriptografia e o `eboot.bin` do jogo seria recusado. Valor legítimo identificado: documentação do mapeamento segmentos-SELF → program headers, que poderia ser portado para C++ com fixtures sintéticas e as mesmas recusas de flag, atribuindo crédito MIT, sem inventar layout.
- **PortPS5 (hard fork GPL-2.0 do AnyPS5):** Projeto maduro (1.282 commits, CI, specs por subsistema) que já implementa a arquitetura alvo deste projeto: relinker ELF→PE com resolução de NIDs, bibliotecas .prx clean-room em ABI SysV e recompiler RDNA2→SPIR-V/Vulkan, sem emulação de CPU. Compatibilidade pública: somente Dreaming Sarah jogável; os demais gate titles ainda não passaram; ASTRO BOT não reportado. Entrada obrigatória: dump do usuário já descriptografado — o gargalo deste projeto permanece externo e intocado. Sem derivação de código, este projeto permanece livre de obrigações GPL; adoção futura de código implicaria GPL-2.0-only.
- **Consequência para o roadmap:** A cadeia crítica inalterada continua sendo: runtime/ABI, libs de sistema, recompiler de shaders e o pipeline raw-ELF incompleto. Próximo passo interno mantido: integrar `ParseElf64Metadata` → `MapElf64LoadSegments` → `ApplyElf64RelativeRelocations` ao fluxo do loader para ELF cru autorizado e, opcionalmente, portar a reconstrução SELF (não criptografado) → ELF com fixtures sintéticas e recusas de flag documentadas.

Etapa — relink real do dump PPSA21567 (AnyPS5)

- **Motivação:** O usuário propôs testar diretamente o ecossistema AnyPS5 (relinker + bibliotecas clean-room), que converte dumps de PS5 em executáveis nativos de Linux/Windows sem emulação de CPU. O projeto AstroPort decidiu *usar* a ferramenta como programa externo validado, sem derivar código (conservando a licença própria livre, sem GPL), documentando o resultado como evidência empírica da cadeia de port.
- **Execução real (02/11/2026):**
  - Download e validação de SHA-256 do binário `relinker-v0.1.4.exe` da release oficial do `boykupovar/AnyPS5` (publicada 2026-10-30): `6f6d49e3613ea7fce859621265068bfd64161a729a8051c83150f2d45ec193e0` — **HASH OK**.

    - `unwrap_self.py` (standalone, MIT/GPL conforme o ecossistema) executado com `--verbose` contra o `eboot.bin` local (259.312.583 bytes): **recusa não aconteceu** — a checagem de entropia dos segmentos PT_LOAD indicou conteúdo *não criptografado* (entropia < 7.5 bits/byte), classificando o dump local como "AnyPS5 Ready": dump já descriptografado fornecido por pipeline externo do usuário.
    - Ambos os módulos `libc.prx` (1.877.882 bytes) e `libSceNpCppWebApi.prx` (7.865.383 bytes) do `sce_module` do dump também desembrulhados com sucesso para ELF.
    - O relinker `--windows --windows-diagnostics` concluiu: **2.930 referências NID analisadas (2930→2930, filtradas 0)**, 2 módulos PRX convertidos (`libc.prx.guest.prx` 2.021.888 bytes, `libSceNpCppWebApi.prx.guest.prx` 8.865.792 bytes), imagem de saída `AstroBot.exe` escrita com 263.828.992 bytes (PE x86-64, base 0x400000 sem ASLR, sem DYNAMIC_BASE).
- **Estado pós-relink (fim de sessão):** O `AstroBot.exe` nativo Windows **foi produzido com sucesso**, com imports resolvidos e 2 módulos guest convertidos. O app0/sce_module com os módulos guest foi criado; faltam as bibliotecas de sistema substitutas em `libs/` (*.prx clean-room) e a cópia de `data/`, `sce_sys/param.json`, `eboot.bin` original em `app0/`.
- **Bloqueio do Windows Defender:** A primeira tentativa de execução de `AstroBot.exe` foi bloqueada pelo Windows Defender ("virus ou software possivelmente indesejado") — falso positivo característico de executável PE não assinado auto-gerado de tamanho grande, base fixa 0x400000 e sem ASLR/DYNAMIC_BASE. De acordo com nossa política, **não mexemos em exclusões do Antivirus**; a decisão de whitelisting/quarantine restore é do usuário.
- **Desaparecimento do upstream (durante a sessão):** O repositório `boykupovar/AnyPS5` ficou inacessível (HTTP 404 no desktop, na página de release e em todos os assets) pouco depois de baixado o `relinker.exe` e testada a conversão. A **biblioteca de sistema** `prx-windows-v0.1.4.zip` (95 MB) não pôde ser baixada. O fork ativo `ymatrix1/PortPS5` (GPL-2.0-only) continua online mas **não tem releases** — exigiria build do zero com MinGW GCC 15.2 pinned + Vulkan SDK + SDL2, sem binários.
- **Proveniência preservada:** A pasta `AnyPS5/` do projeto agora contém o `relinker.exe` validado por hash, o `unwrap_self.py` (GPL-2.0, cópia não alterada) e o script reproduzível `Relinkar_AstroBot.bat`, além das notas de licença e proveniência (`LICENCA_E_PROVENIENCIA.txt`). O upstream desapareceu durante a sessão; recomenda-se preservar estes artefatos.
- **Limites desta validação:** Conversão ≠ jogo rodando. A **execução** do `AstroBot.exe` depende de (a) libs de sistema do AnyPS5/PortPS5, (b) whitelist do Defender decidida pelo usuário, (c) app0 completo (data, sce_sys, eboot.bin original). Sem estas três, o IR do boot não pôde ser observado. ASTRO BOT não tem status de compatibilidade público no AnyPS5; seu status real após o relink é desconhecido até o boot ser tentado.

Etapa — inspetor de SELF no loader próprio (ideias absorvidas do ecossistema, implementação independente)

- **Escopo:** Conforme decisão do usuário ("continuar o meu, aproveitando ideias e soluções"), o conceito de inspeção de tabela de registros SELF (observado no `unwrap_self.py` do AnyPS5) foi **re-implementado do zero em C++** no loader próprio AstroPort — **nenhuma linha de código GPL foi copiada**. A inspeção é puramente metadados: lê a tabela de registros (0x40..0x400, registros de 0x20 bytes: flags, offset de dados, tamanho, tamanho repetido), classifica flags (0x2 criptografado, 0x8 comprimido, 0x800 mapeia-segmento, índice do segmento em flags >> 20) e mede entropia de Shannon (amostra de 64 KiB, limite 7,5 bits/byte). **Nunca descriptografa, descomprime, mapeia ou executa payload.**
- **API adicionada em `src/core/eboot_loader.h`:** `SelfRecordInfo` (por registro: flags, dataOffset/dataSize, hasSegmentMapping, targetSegmentIndex, bitsPerByte, marcadores e validação de limites) e `SelfContainerReport` (magic, contagem do cabeçalho em 0x18, offset do ELF interno, agregados de classificação) + função `InspectSelfContainer()` pública e testável.
- **Integração fail-closed no loader (`ParseSelfHeader`):** um SELF mal-formado agora **recusa a carga** (retorno falso) em vez de seguir adiante às cegas; logs honestos reportam classificação. O `ProcessRelocations()` ganhou o pipeline cru completo para ELF direto (metadados → mapeamento PT_LOAD → relocations R_X86_64_RELATIVE) e segue recusando SELF/G3PROBE antes de tocar em qualquer payload.
- **Testes (`src/test_self_inspection.cpp`, CTest `self_inspection_validation`):** fixtures sintéticos cobrem SELF descriptografado (entropia baixa, flags ausentes, ELF interno localizado em 0x1a0), payload de alta entropia classificado como provavelmente criptografado *sem depender de marco*, flags 0x2+0x8 honrados, magic inválido recusado, tabela mal-formada terminando o parse sem crash, e integração real com `Load()` (SELF → recusa → memória liberada → sem patches/launch). **Suite completa: 6/6 testes passando** (elf_metadata, self_inspection, at9, gameplay_movement, level_collision, software_rasterizer). Build limpo: zero warnings novos, apenas os 3 pré-existentes.
- **Correção de fixture:** o cabeçalho ELF64 interno do fixture de teste passou a ser escrito com larguras nativas corretas (e_type/e_machine u16, e_version/e_phentsize u32, campos u64 nos offsets certos) e o payload movido para 0x350 (respeitando o mínimo validado pelo inspetor). Também removido um segfault latente de acesso a `records[0]` sem guarda.
- **Novo utilitário `diag_eboot`:** CLI de diagnóstico que imprime o informe de inspeção SELF (tabela de registros + entropia por registro), metadados ELF64 internos e o veredito fail-closed do loader para qualquer SELF/ELF — sem escrever nada no arquivo de entrada.
- **Diagnóstico no dump real (leitura apenas):**

    - `eboot.bin` (259.312.583 bytes): **11 registros** parseados (cabeçalho anuncia 12 em 0x18 — o último é o registro de versão fora da tabela, mesmo comportamento observado no unwrap externo), ELF interno em 0x1a0, **zero registros marcados como criptografados/comprimidos**, entropia máxima 6,86 bits/byte (segmento 0, código) — **nenhum payload acima do limite de 7,5**. Isto confirma de forma independente (agora com código próprio) que o dump local está descriptografado ("dumps já preparados").
    - `libc.prx` (1.877.882 bytes): mesmo padrão — 11 registros, ELF interno em 0x1a0, nada marcado criptografado, entropia máxima 6,56 bits/byte.
    - Veredito do loader em ambos: `Load(): recusado` — o pipeline cru inspecciona, registra e **libera a memória**: mapeamento de payload de SELF exige reconstrução autorizada (SELF→ELF), que é a próxima etapa.
- **Política mantida:** a etapa continua apenas-inspeção/metalayout. A reconstrução SELF→ELF (só para containers não criptografados, com recusas explícitas quando qualquer flag 0x2/0x8 estiver presente) é o próximo passo planejado.

Etapa — reconstrução SELF→ELF no loader próprio (dumps não criptografados, fail-closed)

- **Objetivo:** transformar o conhecimento validado na etapa anterior (tabela de registros SELF + entropia) em uma peça funcional do loader: `ReconstructElf64FromSelf()`, implementada do zero em C++ (sem copiar código GPL; conceito de "unwrap" absorvido do ecossistema AnyPS5). Produz um **artefato de análise apenas** — nada é executado, nenhum entry point é chamado, nenhum import é resolvido.
- **Regras de reconstrução implementadas (observadas em dumps reais + validadas por testes):**
    - *Regra 1* — program headers cobertos pela tabela de registros (flag `0x800`, índice em `flags >> 20`): bytes copiados verbatim da posição do registro no container; `p_offset` reapontado para o novo layout (posição congruente com `p_vaddr` módulo page-alignment).
    - *Regra 2* — program headers file-backed NÃO cobertos (PT_GNU_EH_FRAME, PT_GNU_RELRO, PT_TLS, PT_NOTE/PT_DYNAMIC internos): exatidão pela relação vaddr→arquivo do PT_LOAD que os contém ("o arquivo de um PT_LOAD cobre [vaddr, vaddr+filesz) linearmente"), sem duplicação de dados — o phdr apenas reaponta para dentro da cópia do PT_LOAD.
    - *Regra 3* — segmentos restantes sem vaddr, após o último registro coberto (observação real do eboot PPSA21567: o segmento `0x6fffff01` de 0x3d47 bytes fica colado no fim do último registro): copiados em sequência.

    - *Regra 4* — container que termina após o último registro: segmentos de metadados restantes (ex.: PT_NOTE de versão) são **emitidos sem bytes de arquivo** (comportamento documentado), com log honesto; um `PT_LOAD` nessa situação continua sendo **recusado** (fail-closed).
    - Bônus estrutural: `PT_PHDR` tratado (aponta para a própria tabela reconstruída), `e_phoff` movido para 64, section headers zerados (`e_shoff`/`e_shnum`/`e_shstrndx` = 0) e `e_type` PS5 específico (`0xFE10`) normalizado para `ET_DYN` **com log explícito** na cópia de análise.
- **Fail-closed:** recusa reconstrução quando (a) qualquer registro marcado criptografado (0x2) ou comprimido (0x8); (b) qualquer payload mapeado com entropia ≥ 7,5 bits/byte; (c) tamanho do registro ≠ `p_filesz`; (d) registro apontando para segmento inexistente ou duplicado; (e) segmentos file-backed que não podem ser explicados pelas regras; (f) registros/headers fora dos limites do container.
- **Integração no pipeline do loader:** `ProcessRelocations()` agora roda o caminho completo para SELF: inspeção → reconstrução → metadados → mapeamento PT_LOAD → análise de relocations da imagem reconstruída — sempre terminando na recusa honesta (imports/PLT exigem `DT_JMPREL`, ainda não implementado) e liberando memória.
- **Testes (`src/test_self_reconstruction.cpp`, CTest `self_reconstruction_validation`):** fixture de SELF com 5 program headers exatamente como os dumps reais (PT_LOAD coberto + EH_FRAME interno + PT_LOAD coberto + segmento trailing sem vaddr + PT_NOTE sem dados). Verifica layout byte-a-byte, contadores de regras, normalização de e_type, e 5 recusas fail-closed (marcado criptografado, tamanho inconsistente, índice fora da tabela, segmento inexplicado, alta entropia). **Suite: 7/7 testes passando**, build com apenas os 3 warnings pré-existentes.
- **Validação no dump real (leitura apenas):**
    - `eboot.bin`: reconstruído com sucesso — **258.866.503 bytes**, 14 program headers (6 copiados de registros, 6 resolvidos via PT_LOAD, 1 trailing, 1 sem dados), `e_type` `0xFE10` normalizado, entry 0x70. **Auto-validação completa:** nosso próprio parser aceita os metadados (5 PT_LOAD + PT_DYNAMIC) e o mapeamento virtual produz imagem de 270.609.168 bytes (base 0x0, layout vaddr 0x0 / 0x74c0000 / 0x88e0000 / 0x8e5c000 / 0xf4e5410 preservado).
    - Reconstrução casou byte-a-byte com o layout do ELF desembrulhado pela ferramenta externa na sessão anterior (ex.: PT_NOTE em 0xf6d99d8 e PT_DYNAMIC em 0xf6d9a00) — duas implementações independentes convergiram para o mesmo resultado.
    - `libc.prx`: reconstruído — 1.916.954 bytes, mesmo padrão (6+6+1+1 program headers, auto-validação OK, imagem virtual 1.911.096 bytes).
    - Análise de relocations da imagem reconstruída: **DT_JMPREL detectado e recusado por design** — as relocations de PLT exigem resolução de símbolos (imports), que é a

etapa seguinte do projeto.
    - Veredito do loader: `Load()` recusado (honesto) — memória liberada, nada executado.
- **Significado:** o loader próprio agora **entende, valida e reconstrói** containers SELF descriptografados sem depender de ferramenta externa. O gargalo real restante antes de qualquer execução é a resolução de imports/NIDs (PS5: ~2.930 referências no eboot) — nada será executado até que exista uma política completa para imports.

Publicação do estado atual no GitHub

- **Repositório:** `loitalo2-rgb/astro-bot-pc-port` (público), com `main` definido como branch padrão. O repositório antigo (`porteastrobotPC`) deixou de existir (404 — foi renomeado/substituído); o remoto do clone de publicação foi reapontado para o novo endereço.
- **Commits publicados:** `e01ad13` — “AstroPort: inspetor SELF + reconstrucao SELF->ELF, pipeline de analise ELF64, testes 7/7 e relatorios atualizados”; `ddf26fd` — “Adiciona notes de projeto, testes de dispatch/hook e tools de diagnostico”.
- **Conteúdo:** `src/` completo (59 arquivos), `config/`, `memory/`, `tests/hook_demo/`, `tools/`, scripts de build (`build.bat`, `build_clang.bat`, `generate_pdf.bat`, `Rodar_AstroBot_PC.bat`), `CMakeLists.txt`, notas de terceiros e todos os relatórios HTML/PDF (incluindo este).
- **Exclusões deliberadas:** `linkedin/`, `dkb-pc-recomp/`, `referencias/`, `scratch/`, `cache/`, `savedata/`, diretórios de build, binários (`*.exe`/`*.dll`) e qualquer dump do jogo (`eboot.bin`, `*.prx`, `*.sfx`), por escopo e por direitos autorais. O `relinker.exe` permanece fora do repositório (GPL-2.0, verificado por hash na release oficial).
- **README reescrito** em PT-BR com estado do pipeline, regras de reconstrução SELF→ELF, política fail-closed, instruções de build/diagnóstico, licenças de terceiros e o aviso de que nenhum código do jogo é distribuído ou executado.
- **Organização dos arquivos novos encontrados no diretório local:** `dispatch_table.cpp/.h`, `diag_gnf`, `diag_assets` e `test_tex_load` já estão registrados no `CMakeLists.txt`. O `src/injector_main.cpp` é um **protótipo antigo** (hooks em offsets de um executável convertido) que não é compilado nem faz parte do pipeline atual — não foi registrado, para não reintroduzir execução/patches no fluxo de análise.

Análise de reuso — Hookshot (NotRexed/Hookshot, GPL-3.0-or-later)

- **O que é:** recompilação estática nativa do remake de *Ocarina of Time* (Switch 2) para Windows/Linux, sem emulador. Código AArch64 do jogo é traduzido para C (`nx2/lift.py`), compilado junto com uma reimplementação do sistema do console (`host/`) e o trabalho gráfico NVN2 é reproduzido em Vulkan (`gpu/vk/`). Shaders Ampere são rebaixados para Maxwell e traduzidos para SPIR-V pelo recompilador do Eden. Projeto grande (7.220 estrelas, 627 forks, ~15 MB, centenas de testes), construído apenas a partir do dump do próprio usuário — sem jogo, chaves ou executável no repositório.
- **Licença:** GPL-3.0-or-later, incluindo partes de Eden/suyu/Orchid. **Nenhum código será copiado** para o AstroPort; a análise é arquitetural, como já foi feito com o AnyPS5. Conceitos gerais (separação de camadas, disciplina de testes) não são apropriáveis por licença, mas a implementação própria deve permanecer independente.
- **Por que o núcleo do Hookshot NÃO se aplica:** o ponto central dele é recompilar AArch64→C. O alvo do AstroPort (PS5) **já é x86-64**: não há tradução de conjunto de instruções a fazer. Também não há análogo útil para NVN2/Ampere→Maxwell, mesh shaders obrigatórios ou o pipeline de shaders do Switch 2. O que interessa são as *camadas* e os *processos*.
- **Ideias transferíveis (em ordem de valor):**
  - *1. Separação estrita de camadas* — `host/` (serviços do sistema: memória, threads, arquivos, áudio, input, sincronização) vs. código do jogo vs. `gpu/` (API gráfica). O AstroPort já tem `src/kernel`, `src/stubs`, `src/core`, `src/gpu`; a lição é formalizar a fronteira: serviços PS5 atrás de uma tabela de despacho (já iniciada em `dispatch_table`) e renderização atrás de um gravador/decodificador de comandos. Nada do jogo deve chamar Vulkan diretamente.
  - *2. Gravar chamadas gráficas e reproduzi-las depois* — o Hookshot registra as chamadas NVN2 (`host/nvn_native.c`) e as reexecuta no backend Vulkan (`gpu/vk/`). Para o AstroPort, o análogo é um **gravador de command buffers GNM/AGC** que produza um stream inspecionável e reproduzível no nosso backend — permite validar o renderer sem executar o jogo e cria casos de teste reais a partir do dump, em vez de geometria sintética.
  - *3. Censo offline de shaders e assets* — antes de traduzir qualquer coisa, inventariar o que existe (quantos shaders, formatos, variantes, tamanhos). O projeto já tem `shader_recompiler`, `diag_gnf`, `diag_assets` e `test_tex_load`; o próximo passo barato é um censo que classifique os blobs GNF/JXMD do dump e gere um corpus de testes com hash, sem executar nada.
  - *4. Cache/bake de assets com carimbo de versão* — a pipeline `BAKE.md` converte assets para o formato do host de forma reproduzível e versionada. Aplicável ao AstroPort: cache offline de JXMD/GNF convertidos, com hash de entrada/saída e invalidação explícita — evita reprocessar o dump e torna o diagnóstico determinístico.

    - *5. Perfil de funções quentes + PGO* — `scripts/hotlist.py` extrai as funções que cobrem ~85% das amostras e as compila separadamente (`split_hot.py`, `pgo_rt.c`). No AstroPort isso só passa a valer quando houver código de jogo rodando; por ora o equivalente útil é separar os caminhos quentes do *nosso* runtime (mapeamento, resolução de NIDs, decodificação) e medir com benchmark antes/depois.
    - *6. Empacotamento e launcher* — `builder/` com instalador MSI/AppImage, updater e `recomplauncher.json` (manifesto com versão/hash). O AstroPort tem `Rodar_AstroBot_PC.bat` e `settings.ini`; quando houver distribuição, vale um manifesto com hash e verificação, no mesmo espírito.
    - *7. Disciplina de testes e documentação por modo de falha* — centenas de testes pequenos (C++, C, Python) executados por `run_all.py` e **um documento em `docs/` por falha investigada** (device loss, swapchain, sampler, jitter temporal…). É a ideia mais barata e mais valiosa: manter a suíte CTest crescendo e registrar cada recusa/falha em uma nota curta com sintoma, causa e decisão.
- **O que não aproveitar:** lifter AArch64→C, decodificadores NSO/NCA/NSP do Switch 2, camada NVN2, rebaixamento Ampere→Maxwell, shaders FSR/DLSS do projeto, instaladores MSI/AppImage e todo o código GPL. Nada disso corresponde ao alvo PS5 x86-64 nem pode ser relicenciado.
- **Recomendação:** adotar agora as ideias 1, 2, 3 e 7 (camadas formalizadas, gravador de comandos gráficos, censo offline de shaders/assets e nota de falha + teste por sintoma). As ideias 4 e 5 entram junto com o cache de assets e com o primeiro código de jogo executável; a 6 só na etapa de distribuição. O Hookshot confirma que a arquitetura do AstroPort está na direção certa — o que falta nele (tradução de ISA) é exatamente o que o PS5 não precisa.

Etapa — censo offline de assets (leitura apenas, base para o renderer e para o cache)

- **Motivo:** aplicar a ideia 3 do Hookshot (censo antes de traduzir) sem executar o jogo. Até agora o projeto conhecia *seis* malhas JXMD do hub; o dump tem dezenas de milhares de containers. O censo mede o que existe de fato, com que formatos, tamanhos e variantes, para dimensionar decodificadores e corpus de teste.
- **Implementação (própria, C++):**
    - `src/assets/asset_census.*` — classificação por cabeçalho fixo de GNF, JXMD, SELF, ELF64 e SPIR-V; valida limites contra o tamanho real do arquivo; nunca lê payload.
    - `src/assets/gnf_format.*` — tabela central de formatos GNF→Vulkan, compartilhada pelo loader e pelo censo (impede que os dois divirjam).

    - `src/diag_census.cpp` — percorre a árvore, lê no máximo 96 bytes por arquivo, agrega por container/formato/extensão e opcionalmente grava CSV (`path,size,kind,valid,detail`).
    - `src/test_asset_census.cpp` — CTest `asset_census_validation`: cabeçalhos sintéticos GNF v4/v5 (BC4, BC7, formato não mapeado, truncado/fora do arquivo), JXMD válido e com chunk além do EOF, ELF64, SELF, SPIR-V, dados desconhecidos e entradas degeneradas.
- **Resultado no dump real PPSA21567 (somente leitura):**
    - **166.717 arquivos**, **148,83 GiB**, **0 ilegíveis**; 105 s na primeira passada (cache frio) e 56 s na segunda.
    - **GNF:** 28.949 texturas, 110,20 GiB, **todas com cabeçalho válido**; versões v4 = 140, v5 = 28.809; dimensões de 4×2 até 4172×4096; payload somado de 111.067 MiB; até 13 níveis de mip; tile modes 5 (472) e 9 (28.477) — ou seja, **todas as texturas são tiled**, o que valida o caminho de decodificação por blocos já existente.
    - **Distribuição de formatos:** 0xCAF0 = 13.202 (BC4), 0xCB60 = 9.695 (BC7 sRGB), 0xCB10 = 4.847 (BC7 UNORM, normais), 0xCB50 = 719 (BC7 UNORM, máscaras), 0xCB30 = 457 (BC7 sRGB), 0xCAA0 = 2 (BC1 sRGB, nome do arquivo confirma `*_bc1s`), 0x8B60 = 3 (BC7 sRGB), 0x8AF0 = 1 (BC4).
    - **23 texturas em 7 IDs sem decodificador** (0x0B50=1, 0x0B60=4, 0x4AF0=3, 0x4B50=1, 0x4B60=4, 0xC380=3, 0xCA90=7) — 0,08% do total.
    - **JXMD:** 15.503 containers (26,18 GiB) — mais que os 15.333 arquivos `.jxm`, porque **170 arquivos `.jmt` também são JXMD** (confirmado pelo magic, não pela extensão). Strides: 48 = 14.866 (dominante), 36 = 620, 40 = 16, 32 = 1; soma de elementos de chunk = 215.783.
    - **Outros volumes:** `.anim` 61.160 (1,86 GiB), `.odxb` 19.957 (55 MiB), `.szd` 12.705 (3,72 GiB), `.pfbhk` 8.755 (3,63 GiB), `.skel` 7.346, `.jpx` 3.476, `.lvx` 2.474 — candidatos naturais ao próximo censo (áudio, animação, esqueleto, nível).
    - **SELF:** 3 containers no dump (eboot + 2 PRX), coerente com a etapa de reconstrução.
- **Mudança de comportamento fail-closed:** antes, `LoadGnf` rotulava qualquer formato não-BC4 como BC7 UNORM — um default silencioso que decodificaria lixo com o bloco errado. Agora ele consulta a tabela central e **recusa** IDs desconhecidos com log explícito (23 texturas afetadas, 0,08%), em vez de adivinhar. Formatos confirmados: BC4 (0xCAF0/0x8AF0), BC7 UNORM (0xCB10/0xCB50), BC7 sRGB (0xCB30/0xCB60/0x8B60) e BC1 sRGB (0xCAA0).
- **Testes:** `asset_census_validation` adicionado; suíte agora em **8/8 testes passando**, build sem novos warnings.
- **Próximo uso do censo:** (a) gerar um corpus de teste por formato a partir de arquivos reais, sem executar o jogo; (b) dimensionar o decodificador de `.szd`/`.anim`/`.skel`; (c) servir de base

para o gravador de comandos gráficos (ideia 2 do Hookshot), que precisa saber quais texturas e formatos os draws referenciam.

Etapa — validação de layout GNF e corpus de teste por formato (leitura apenas)

- **Objetivo:** transformar o censo em garantia de decodificação. Antes de gerar o corpus, era preciso saber se o número de bytes declarado no GNF confere com a cadeia de mips calculada a partir do cabeçalho — e qual regra de tiling o dump realmente usa.
- **Implementação:** `src/assets/gnf_layout.*` calcula, só com o cabeçalho, três hipóteses de tamanho (linear, cada mip alinhado a 64 KiB, e total arredondado a 64 KiB), além da geometria de tile (128×64 blocos para BC4, 64×64 para BC7/BC1). O mesmo cálculo de tile foi centralizado e o loader passou a usá-lo (antes era uma cópia local). `src/diag_gnf_validate.cpp` percorre o dump, valida cada GNF e escreve um manifesto de corpus.
- **Resultado nos 28.949 GNF reais (9,5 s, 0 cabeçalhos inválidos):**
  - Payload declarado: **116.414.251.008 bytes**; soma linear dos mips: **115.134.925.248 bytes**.
  - **Invariante confirmado em 100% dos arquivos: o payload declarado nunca é menor que a soma linear dos mips** (nenhum “menor” em nenhum formato). O excedente médio é ~1,1% e representa padding/arranjo de tiles.
  - **Hipóteses testadas e descartadas:** “cada mip ocupa tiles inteiros de 64 KiB” (o declarado é menor em 28.900 arquivos, então mips pequenos não são arredondados individualmente) e “cada mip começa em fronteira de 64 KiB” (também não).
  - **Hipótese mais próxima:** “mips contíguos + payload total arredondado a 64 KiB” — exata em 13.179/13.202 BC4 (99,8%), 4.835/4.847 BC7 UNORM, 713/719 CB50 e 9.177/9.695 BC7 sRGB, mas apenas 76/457 no formato 0xCB30. Ou seja: é a regra dominante, não universal; a regra exata de padding por formato ainda não foi totalmente derivada e **não será assumida**.
  - Nenhum arquivo tem payload passando do fim do container (`pastEof = 0`) — a leitura linear dos mips está sempre dentro do arquivo.
- **Correção de rumo registrada:** uma primeira alteração no loader passou a avançar os mips pelo tamanho “tileado”; o validador mostrou que isso quebraria 28.900 arquivos (o payload real é menor que essa soma). A alteração foi **revertida** e substituída por duas verificações conservadoras que o dado real sustenta: (a) recusar payload que ultrapassa o fim do arquivo; (b) recusar payload menor que a soma linear dos mips. Nenhuma suposição sobre padding é usada para decodificar.

- **Corpus de teste (sem copiar bytes do jogo):** o validador gera um manifesto CSV com **71 representantes**, um por combinação (formato, tile mode, versão, número de mips), contendo caminho, tamanho, formato, dimensões, payload, as três estimativas e **FNV-1a 64 dos primeiros 1 MiB** de cada arquivo. O manifesto permite reencontrar/validar os mesmos dados localmente sem versionar conteúdo proprietário — nenhum arquivo do dump entra no repositório.
- **Testes:** `asset_census_validation` ganhou casos de geometria de tile (BC4 128×64, BC7 64×64), textura 4×4 BC4 ocupando um tile inteiro, 128×64 BC7 em um tile, 4096×4096 BC7 com 256 tiles exatos, layout linear sem padding, alinhamento do segundo mip e total arredondado. Suíte segue **8/8**, build sem novos warnings.
- **Próximo uso:** o manifesto é a base do corpus do renderer; com ele é possível abrir um subconjunto representativo, decodificar mips de verdade e comparar com o upload Vulkan — sem executar o jogo e sem inflar o repositório.

Próximos Passos Estratégicos

1. **Integração das relocations ao pipeline de ELF cru:** Conectar o fluxo `ParseElf64Metadata` → `MapElf64LoadSegments` → `ApplyElf64RelativeRelocations` ao caminho do loader para ELF cru de autoria/uso autorizado, substituindo a interrupção hardcoded em `ProcessRelocations()`, com logs honestos por etapa. Relocations simbólicas/PLT continuam dependentes de `DT_SYMTAB/DT_STRTAB` e de uma tabela de imports autorizada do jogo.
2. **SELF e runtime do jogo:** Estudar somente interfaces e documentação autorizadas para o formato e runtime. Não contornar criptografia/controles de acesso; até haver um fluxo permitido e validado, manter o `eboot.bin` em modo de inspeção, sem patches ou transferência de controle.
3. **Validação Audiodec no jogo:** Obter uma tabela de imports ligada ao ASTRO BOT e uma fixture de áudio descompactada com frame ATRAC9 válido; confirmar NIDs/ABI e verificar PCM na saída WASAPI. Até isso existir, manter o áudio do jogo como não validado.
4. **Gameplay & Colisão:** Encontrar/decodificar a fonte de geometria ausente dos seis JXM e recuperar mais malha autêntica do hub; substituir o pipeline GDI/software por renderização acelerada com materiais/texturas; repetir o benchmark CSV com baseline idêntico antes/depois de cada otimização e em durações maiores; validar a retomada Vulkan ao alternar abas e movimento com controle, contatos em cantos, quedas e transições em runtime. A cobertura do grid sintético/local não substitui testes visuais interativos com controle.